

Table for Four: An Agentic Restaurant Concierge

CMU AI Agent Certification · Capstone Final Report

Author: Manish Bhatt

Table for Four takes one sentence, "somewhere Italian for four on Friday", and carries it to a confirmed reservation with a person's approval on record. This report covers the problem, the architecture, how the design changed, how it was evaluated, and where a person must still decide.

1. The problem, and who has it

Booking dinner for a group is a small decision that ends in a real commitment. One request has to settle cuisine, area, party size, time, budget, dietary needs and what is free that night, and then act on the answer. The intended user is whoever ends up organising dinner for everyone else. It suits an agent because the data is live, the preferences are personal and worth carrying between visits, and the job ends in an action rather than an answer.

That last property is why it needs care. A search can be run again; a booking cannot be undone. It holds a table and commits people to be somewhere at a time, so the failure here is not a weak paragraph but a guest arriving at a restaurant that has never heard of them.

2. Goal, scope and constraints

The goal is one sentence in, a booked table out, with the reasons visible and a person's approval on record. Working well means five things: the right restaurant near the top of the shortlist with the reason it fits; nothing in a reply that a tool did not return; no booking until a person approves it; a record afterwards of who did what and what that person was shown; and the whole thing still running with no API key.

Each boundary was chosen rather than inherited. The agent handles dining only. Bookings run against a mock reservation service written for this project, because the real ones are partner gated. Partner offers are synthetic and labelled as such. There are no payments and no logins.

The constraint that shaped the most code was offline operation. Every external dependency has a stored fallback, so a missing key degrades the experience instead of ending it, and the test suite runs with no key at all, which exercises that path on every run.

3. The system, as one integrated whole

Capability sits behind tool boundaries and authority sits behind declared permissions. Four Model Context Protocol servers expose seven tools: a search server calling the Google Places API; an offers server ranking partner perks with a vector store and a locally run embedding model, weighing how well an offer matches the request against whether it can be used at all that day; a web server fetching cited menu highlights; and a booking server, a FastAPI service with its own database, which owns the twenty four hour cancellation rule.

Two paths drive those servers. The scripted path is an explicit LangGraph state machine: parse, search, match offers, rank, propose, approve, book, record, with one bounded retry when the first search was too narrow. The conversational path is event driven, because a real conversation has no fixed order.

Between them sits what makes this one system rather than a pile of tools. Every tool call runs as a declared unit, and the registry refuses anything that unit was not granted. Five units are split by authority rather than skill: a host, one that finds tables, one that researches food, one that keeps the member's record, and one that books. The host is the only unit that runs a model and holds no tools at all, so it can only ask a unit that does. Misbehaviour is therefore an exception rather than a policy violation, and the unit that books cannot search, so a booking can only name a restaurant genuinely found.

Governance cuts across all of it, and is section 7.

4. How the design evolved

The through line across six modules: every rule written into the prompt broke sooner or later, and the same rule written into the code held.

Module one put search behind a tool boundary with an offline copy of the data, so the demo never depended on a key. Module two replaced free choice of tools with an explicit state machine and added memory in three tiers. Module three added retrieval over the offers store with a labelled test set, so relevance became measured rather than guessed.

Module four was the turning point, and it included a decision not to build something. Tree search over generated branches was rejected for the reasoning layer: the branches here are retrieved rather than reasoned, the hard constraints have exact answers from tool calls, and the guest is the search algorithm by design. The permission harness was built instead. Module five redrew the unit boundaries by authority rather than skill, which is where the host lost its tools. Module six added governance.

One refinement generalises. An availability result used to carry every open time, with the model instructed not to read the others back once the guest had chosen. That instruction failed. Removing the other times from the payload worked immediately. Where an instruction and a data shape disagree, the data shape wins.

5. Implementation

The project is Python, managed with uv, each server owning the data it uses. Orchestration is LangGraph with a checkpoint, which is what makes a genuine pause at the approval step possible. Retrieval is Chroma with a locally run embedding model, and the booking service is FastAPI over SQLite. The conversational path uses a small, inexpensive OpenAI chat model through LangChain, with deterministic logic taking over when no key is present. Three surfaces sit on top: a Streamlit app, a terminal chat, and LangGraph Studio.

6. Evaluation, results and limitations

Does the shortlist contain the right restaurant? Ten labelled requests, with the correct answers recorded in advance and scored on a fixed date so the expiry filter behaves identically every run. A relevant restaurant appeared in the top three in every case, and first in every case. Precision at three is lower, at 0.533, and is reported rather than optimised: relevance is labelled per restaurant while the search returns individual offers.

Does the agent behave as designed? There are 236 automated tests, each named for the behaviour it guards, all passing offline in about thirty seconds.

Does it invent things? Every reply is compared against what the tools returned, and a time, date, confirmation number or email address that no tool produced is removed before the guest reads it.

Does it stay cheap? Two model calls per turn, bounded at six tool steps, with a test pinning the tool description budget. Splitting the system into five units added no model calls at all.

The strength is that the guarantees sit at chokepoints rather than spread across call sites, and that everything runs offline. The limitations are named rather than hidden: the web lookup has not been tested against a hostile page, the largest open surface; the reply check does not cover dish or restaurant names, which would need entity extraction and a judgement; a booking cancelled outside the app leaves a stale line in the guest's history; and the database holds one guest at a time. The design also handles authority far better than complex planning: an evening across three venues, where travel time interacts with closing hours, is a different problem.

7. Safety, reliability and human oversight

The rule is that the agent may look things up on its own, but anything it cannot undo waits for a person.

Guardrails constrain what is possible rather than what is encouraged. The booking handler refuses without an email, a matching shortlist entry, a date not in the past, a party size the guest actually gave, and a time the availability check genuinely offered. Standing preferences change only on the guest's own words. The cancellation rule is enforced by the booking service, so the agent relays a refusal verbatim and cannot claim a cancellation that never happened.

Oversight is triggered by the class of action, not the model's confidence in it. Every booking stops: the scripted path checkpoints and genuinely halts until a caller resumes it, and the conversational path puts the summary on screen with Reserve and Change my mind. Anything that is not clearly a yes counts as a no, and nothing books unattended except under a flag a person sets in advance. Confidence thresholds were avoided deliberately, because a threshold is a number the model chooses and the failure actually observed is confident invention.

Reliability comes from the same place: stored fallbacks everywhere, bounded loops, and an audit writer that swallows its own errors, because a governance log must never be the reason a guest does not get a table.

8. Repository

github.com/manishbhatt19/table-for-four

The repository holds a README covering the problem, architecture, setup and usage; the full source; the offline test suite; the evaluation script; and this report alongside the presentation. No keys are needed to run it.